Multi-Agent Reinforcement Learning for Adversarial Spatial Decision-Making with Temporal Constraints

Chaitanya Chaudhary
Dept. of Computer Science & Engineering
IIT Gandhinagar
chaitanya.chaudhary@iitgn.ac.in

Dhruv Meshram
Dept. of Computer Science & Engineering
IIT Gandhinagar
dhruv.meshram@iitgn.ac.in

Abhishek Tagalpallewar
Dept. of Mechanical Engineering
IIT Gandhinagar
abhishek.tagalpallewar@iitgn.ac.in

Vedant Bondekar
Dept. of Computer Science & Engineering
IIT Gandhinagar
vedant.bondekar@iitgn.ac.in

September 27, 2026

Abstract

Multi-agent reinforcement learning (MARL) studies sequential decision-making problems in which multiple learning agents interact within a shared environment, making strategic adaptation central to the learning process. In competitive settings, the behavior of each agent depends not only on the environment but also on the evolving policy of an opposing agent, creating a natural connection between reinforcement learning and game-theoretic decision-making. In this work, we investigate these dynamics using **Modified Wallz**, a deterministic, sequential, two-player zero-sum environment in which agents simultaneously pursue opposing objectives while modifying the spatial constraints of the board. The environment extends the original wall-based mechanics with reusable walls, a one-time wall-bypassing hop, and temporary cell blocking, introducing resource constraints, temporal dependencies, and dynamically changing action availability. These mechanisms provide a controlled setting for studying how strategic resources and evolving constraints affect multi-agent policy learning. We formulate the environment in terms of agent observations, actions, rewards, and state transitions, and propose to evaluate MARL agents against heuristic and search-based baselines. For smaller configurations, exhaustive game-tree analysis and minimax-based reasoning will be investigated as references for learned behavior and computational feasibility. By varying resource and temporal parameters, we aim to characterize their effects on strategic adaptation, learning performance, and computational complexity, thereby providing insight into multi-agent decision-making under dynamic spatial and resource constraints.

Keywords: First keyword Second keyword More

1 Introduction

Multi-Agent Reinforcement Learning (MARL) considers sequential decision-making problems in which multiple adaptive agents interact within a shared environment. Such settings arise whenever the outcome of an agent's actions depends on the behavior of other decision-makers, making learning fundamentally different from conventional single-agent reinforcement learning. Markov games provide a natural mathematical framework for these settings by allowing multiple agents with interacting or competing objectives to influence a common environment [1]. MARL has consequently developed strong connections with game theory, optimization, and reinforcement learning, with applications spanning cooperative, competitive, and mixed-motive environments [2, 3].

A central difficulty in MARL is that agents learn simultaneously. From the perspective of an individual learner, the effective environment can therefore become non-stationary as the policies of other agents change during training [4, 3]. In addition, reasoning over multiple agents introduces joint action spaces whose size grows with the number of decision-makers, increasing the computational burden of policy search. These issues are particularly relevant in competitive settings, where an agent must learn not only how to maximize its own objective but also how its decisions affect, and are affected by, an adaptive opponent. Consequently, strategic adaptation rather than isolated action optimization becomes an important component of the learning problem.

We investigate these issues using **Modified Wallz**, a deterministic, sequential, two-player, zero-sum board environment. Each player attempts to reach the opponent's side while moving a pawn and placing walls that restrict movement subject to path-preservation constraints. The modified environment introduces reusable walls, a one-time opportunity to hop over an opponent's wall under defined legal conditions, and temporary cell blocking for a fixed number of future moves. These mechanisms introduce resource constraints and temporal dependencies into a fully observable competitive environment, allowing strategic interaction to be studied while keeping the underlying state and transition structure explicit. A playable prototype of the modified Wallz game is available online for demonstration and experimentation: `https://tagalpallewarabhishek.github.io/wallz-custom-game/`.

The resulting environment provides a controlled setting for investigating how adaptive opponents, reusable strategic resources, and dynamically changing spatial constraints influence policy learning. We formulate the environment as a multi-agent sequential decision problem and aim to evaluate MARL agents against heuristic and search-based baselines. For smaller configurations, exhaustive game-tree analysis and minimax reasoning will also be considered to provide references for learned policies and to examine the tractability of exact computation. By varying the resource and temporal parameters, we seek to characterize their effects on strategic behavior, learning performance, and computational complexity. This motivates the formal definitions of the environment, agents, actions, rewards, and state transitions developed in the following sections.

2 Preliminaries

This section establishes the theoretical foundations, standard multi-agent reinforcement learning (MARL) concepts, game-theoretic definitions, and graph notations that underpin the formalization of Modified Wallz and its evaluation baselines.

2.1 Multi-Agent Reinforcement Learning

In reinforcement learning (RL), an autonomous *agent* interacts sequentially with an external *environment* [5]. At each discrete interaction step, the environment is characterized by a *state* $s \in \mathcal{S}$ capturing the current physical and informational configuration of the system. An agent receives an *observation* o, which reflects the perceptual information accessible to it; when the domain is fully observable, the observation is identical to the complete state ($o = s$).

Based on its observation, the agent selects an *action* a from its allowable action set $\mathcal{A}$. The execution of this action causes the environment to transition to a subsequent state $s' \in \mathcal{S}$ and emit a scalar *reward* R providing feedback regarding the agent's performance. The agent's decision-making behavior is governed by a *policy* π, which specifies a mapping from states to actions, either deterministically ($\pi : \mathcal{S} \to \mathcal{A}$) or as a probability distribution over actions ($\pi : \mathcal{S} \to \Delta(\mathcal{A})$).

In *multi-agent* reinforcement learning, multiple agents operate concurrently within a shared environment [2]. Because state transitions and reward payoffs depend on the joint actions of all participating decision-makers, the effective environment becomes non-stationary from the vantage point of any individual learner as the policies of other agents co-evolve during training [4, 3]. Consequently, learning in multi-agent settings requires strategic adaptation to opposing policies rather than isolated optimization against a stationary background.

2.2 Markov Games

Sequential decision-making with multiple interacting agents is formally described by the framework of *Markov games* (also known as stochastic games), introduced as a multi-agent extension of Markov Decision Processes [1]. Formally, an n-player discrete-time Markov game is defined by the tuple:

$$\langle \mathcal{N}, \mathcal{S}, (\mathcal{A}_i)_{i\in\mathcal{N}}, T, (R_i)_{i\in\mathcal{N}}, \gamma \rangle, \tag{1}$$

where:

- $\mathcal{N} = \{1, \ldots, n\}$ is the finite set of interacting agents.
- $\mathcal{S}$ is the set of all valid environment states.
- $\mathcal{A}_i$ denotes the action space of agent i, with $\mathcal{A}_i(s) \subseteq \mathcal{A}_i$ designating the state-dependent legal action subset in state $s \in \mathcal{S}$.
- T specifies the transition dynamics mapping the current state and joint actions to the subsequent state.
- $R_i : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the reward function quantifying the payoff of agent i.
- $\gamma \in [0, 1]$ is the discount factor for future returns.

While general Markov games may involve joint actions, the Modified Wallz instance considered in this work is turn-based: at each decision step, only the currently active player selects and executes a legal action, and the resulting transition is therefore determined by the current state and that active player's action.

In competitive multi-agent environments, agents pursue opposing objectives. In two-player interactions ($|\mathcal{N}| = 2$), the game is *zero-sum* when payoffs satisfy $R_1(s, a, s') + R_2(s, a, s') = 0$ for all transitions, signifying strictly conflicting interests.

2.3 Sequential Zero-Sum Games

A sequential (or turn-based extensive-form) game structures decision-making through alternating turns rather than simultaneous action execution. At each discrete time step $t \in \mathbb{N}_0$, an active turn indicator $z_t \in \mathcal{N}$ uniquely designates the active player, while all non-active players remain passive.

The environment exhibits *deterministic transitions* if, given the current state s_t and the active agent's legal action $a_t \in \mathcal{A}_{z_t}(s_t)$, the successor state s_{t+1} is uniquely determined by a deterministic transition function $s_{t+1} = T(s_t, a_t)$. A game has perfect information when every player has complete, unoccluded access to the current environment state, with no hidden state variables or stochastic chance events. A sequential two-player zero-sum game of perfect information is strictly adversarial: any strategic advantage secured by one player corresponds identically to an equivalent deficit for the opposing adversary.

2.4 Minimax and Backward Induction

Two-player zero-sum games of perfect information are governed by the classical minimax theorem of game theory. Under minimax reasoning, each player assumes that the adversary will execute the optimal counter-strategy to minimize the player's return. The maximizing agent selects the action satisfying:

$$V_1^*(s) = \max_{\pi_1} \min_{\pi_2} \mathbb{E}_{\pi_1,\pi_2}[R_1 \mid s] = \min_{\pi_2} \max_{\pi_1} \mathbb{E}_{\pi_1,\pi_2}[R_1 \mid s]. \tag{2}$$

For finite extensive-form games without cycles, *backward induction* provides an exact algorithm to determine subgame-perfect Nash equilibria by recursively propagating value evaluations from the terminal leaf nodes of the game tree back to the initial root state.

When exhaustive tree traversal is computationally intractable due to combinatorial state growth, *depth-limited minimax search* evaluates paths up to a specified lookahead horizon D, truncating further expansion and estimating non-terminal state values using a domain-specific heuristic evaluation function. To reduce computational complexity, *Alpha-Beta pruning* dynamically maintains branch bounds $[\alpha, \beta]$ to eliminate provably suboptimal subtrees without altering the resulting minimax decision.

2.5 Grid and Shortest-Path Representation

Discrete board games defined over spatial cells are naturally modeled as topological graphs $G = (V, E)$. The vertex set V corresponds to the collection of grid cells C, and the undirected edge set $E \subseteq V \times V$ represents allowable orthogonal transitions between neighboring cells (u, v) satisfying $\|u - v\|_1 = 1$.

Static or temporary obstacles modify the connectivity of this graph by removing edges from E (such as wall placements that obstruct cell boundaries) or temporarily removing vertices from V (such as blocked cells). This induces an active traversable subgraph $G_t = (V_t, E_t)$ at time step t. The geodesic distance $d(u, v; G_t)$ denotes the minimum number of edge traversals required to connect cell u to cell v within G_t, computable in polynomial time via standard graph traversal algorithms (e.g., Breadth-First Search). A configuration satisfies path preservation if $d(p_i, G_i; G_t) < \infty$ for all players $i \in \mathcal{N}$, guaranteeing that a legal path to the goal region always remains open.

2.6 Summary of Notation

Table 1 provides a concise summary of the primary mathematical symbols used repeatedly throughout this paper.

Table 1: Summary of primary mathematical notation.

Symbol	Description
N	Board dimension ($N \times N$ discrete grid)
C	Set of all valid board cells, $C = \{(x, y) \in \mathbb{Z}^2 \mid 1 \le x, y \le N\}$
$i, -i$	Active player index $i \in \{1, 2\}$ and opposing player index $-i = 3 - i$
p_i	Current pawn position of player i, $p_i = (x_i, y_i) \in C$
G_i	Designated goal region for player i, $G_i \subset C$
$\mathcal{W}$	Universe of all legally specifiable wall placements on the grid
$W_t, W_{i,t}$	Set of all active walls at step t, and subset placed by player i ($W_t = W_{1,t} \cup W_{2,t}$)
m_i	Remaining unused wall inventory for player i ($m_i \in \{0, 1, \ldots, M\}$, with $M = 10$)
r_i	Pointer to player i's currently replaceable (most recently placed) wall ($r_i \in W_{i,t} \cup \{\varnothing\}$)
h_i	Binary hop availability flag for player i ($h_i \in \{0, 1\}$)
τ_t	Temporary-block duration mapping, $\tau_t : C \to \{0, 1, \ldots, K\}$
K	Predefined fixed duration parameter for temporary cell blocking
z_t	Turn indicator identifying the active player at time step t ($z_t \in \{1, 2\}$)
T	Deterministic state transition function defined over legal state-action pairs
R_i	Zero-sum scalar reward function for player i

3 Problem Formulation

3.1 Game Definition

We formalize **Modified Wallz** as a discrete-time, two-player, sequential zero-sum Markov game of perfect information with deterministic transitions:

$$\mathcal{G} = \langle \mathcal{N}, \mathcal{S}, (\mathcal{A}_i)_{i \in \mathcal{N}}, T, (R_i)_{i \in \mathcal{N}} \rangle, \tag{3}$$

where:

- $\mathcal{N} = \{1, 2\}$ denotes the set of players. For an active player $i \in \mathcal{N}$, the opposing player is denoted by $-i = 3 - i$.

- $\mathcal{S}$ is the finite set of valid, fully observable environment states.
- $\mathcal{A}_i$ denotes the action space available to player i, with $\mathcal{A}_i(s) \subseteq \mathcal{A}_i$ representing the state-dependent legal action subset in state $s \in \mathcal{S}$.
- $T : \big\{(s,a) \;\big|\; s \in \mathcal{S},\ a \in \mathcal{A}_{z(s)}(s)\big\} \to \mathcal{S}$ is the deterministic state transition function defined over legal state-action pairs, where $z(s) \in \mathcal{N}$ denotes the active player in state s.
- $R_i : \mathcal{S} \times \mathcal{A}_i \times \mathcal{S} \to \mathbb{R}$ is the reward function for player i, satisfying the zero-sum condition $R_1(s,a,s') + R_2(s,a,s') = 0$ for all transitions.

The game proceeds sequentially by strict turn alternation: at each discrete time step $t \in \mathbb{N}_0$, exactly one player $z_t \in \mathcal{N}$ selects an action from their legal action set $\mathcal{A}_{z_t}(s_t)$, while the other player remains inactive. The environment is fully observable, deterministic, and possesses no private information or stochastic transitions.

3.2 Board and Player State

The game board is modeled as an $N \times N$ discrete grid, where $N \in \mathbb{N}_{\geq 3}$ is a configurable dimension parameter. The set of valid grid cells is defined as:

$$C = \big\{(x,y) \in \mathbb{Z}^2 \;\big|\; 1 \leq x, y \leq N\big\}, \tag{4}$$

where $c = (x,y) \in C$ indexes cell coordinates with x denoting the horizontal column and y denoting the vertical row.

Each player $i \in \mathcal{N}$ controls a single pawn occupying a position:

$$p_i = (x_i, y_i) \in C. \tag{5}$$

The pawns occupy distinct cells at all valid states, enforcing the mutual exclusion invariant $p_1 \neq p_2$.

Players begin at designated initial coordinates situated on opposing sides of the board. The goal region for player i is denoted by a non-empty subset $G_i \subset C$. Without loss of generality, in a standard orientation:

$$G_1 = \big\{(x,y) \in C \;\big|\; y = N\big\}, \quad G_2 = \big\{(x,y) \in C \;\big|\; y = 1\big\}. \tag{6}$$

Player i achieves victory when their pawn enters any cell within G_i. The turn indicator variable is denoted by $z_t \in \mathcal{N}$, identifying which agent is active at time step t.

3.3 Wall Representation

Walls represent static structural obstructions placed along the internal edges separating adjacent cells, thereby inhibiting orthogonal pawn traversal. Let $\mathcal{W}$ denote the finite universe of all possible wall placements on the $N \times N$ grid. Each wall placement $w \in \mathcal{W}$ is formally defined as:

$$w = (c_{\text{wall}}, o), \tag{7}$$

where c_{wall} denotes the reference grid location of the wall boundary and $o \in \{\text{H}, \text{V}\}$ specifies horizontal or vertical orientation. While standard implementations define walls spanning two adjacent cell edges, the specific edge-indexing convention is abstracted into $\mathcal{W}$ without affecting the mathematical formulation.

At any time step t, the set of all active walls currently present on the board is denoted by $W_t \subset \mathcal{W}$. The total wall configuration is partitioned into the disjoint subsets of walls erected by each player:

$$W_t = W_{1,t} \cup W_{2,t}, \quad \text{with } W_{1,t} \cap W_{2,t} = \emptyset, \tag{8}$$

where $W_{i,t}$ denotes the set of active walls placed by player i. To ensure physical validity, no two walls in W_t may overlap or intersect.

Crucially, wall placements must adhere to the fundamental path-preservation constraint of Wallz. Let $\text{Adj}(C, W_t, \tau_t)$ denote the undirected connectivity graph over board cells induced by walls W_t and temporary obstructions. We define the path-validity predicate:

$$\text{PathExists}(s,i) \in \{0,1\}, \tag{9}$$

which evaluates to 1 if and only if there exists at least one unblocked sequence of adjacent traversable cells in $\mathrm{Adj}(C, W_t, \tau_t)$ connecting player i's current position p_i to some goal cell $g \in G_i$. A wall placement or modification is strictly legal only if it preserves paths for *both* players:

$$\mathrm{PathExists}(s', 1) = 1 \quad \text{and} \quad \mathrm{PathExists}(s', 2) = 1. \tag{10}$$

Thus, no action may completely trap or isolate either agent from their respective target region.

3.4 Reusable Wall State

The first modification introduces reusable walls under a restricted replacement policy. Each player begins the game with an initial inventory of $M = 10$ walls. The remaining unused wall inventory for player i at time t is denoted by:

$$m_i \in \{0, 1, \ldots, M\}. \tag{11}$$

Unlike unconstrained wall repositioning, reusability is strictly constrained by recency:

Only the most recently placed wall belonging to a player can be repositioned.

To track this capability, we associate with each player $i \in \mathcal{N}$ a replaceable wall state variable:

$$r_i \in W_{i,t} \cup \{\varnothing\}, \tag{12}$$

where $r_i = \varnothing$ indicates that player i has not yet placed any wall (or currently has no active replaceable wall), while $r_i = w \in W_{i,t}$ uniquely identifies the single wall most recently placed by player i. Walls placed earlier than the most recent one become permanent fixtures for the remainder of the match.

This mechanic defines two distinct wall-related action categories:

1. **New Wall Placement:** An action placing a previously unused wall $w \in \mathcal{W} \setminus W_t$. This action is permissible only when $m_i > 0$, consumes one inventory unit ($m_i' = m_i - 1$), adds w to $W_{i,t}$, and designates $r_i' = w$.
2. **Wall Replacement (Repositioning):** An action that removes the player's active replaceable wall r_i from the board and simultaneously places a new legal wall $w' \in \mathcal{W} \setminus (W_t \setminus \{r_i\})$. This action is available only if $r_i \neq \varnothing$. Under this rule, repositioning reuses the existing wall and therefore does not consume an additional wall from the player's finite inventory ($m_i' = m_i$), while updating the replaceable wall identifier to $r_i' = w'$.

3.5 One-Time Hop State

The second modification grants each player a single, non-recoverable opportunity per game to hop over an intervening opponent's wall. This resource is represented for each player $i \in \mathcal{N}$ by a binary variable:

$$h_i \in \{0, 1\}, \tag{13}$$

where:

- $h_i = 1$ denotes that player i's hop action is unused and currently available,
- $h_i = 0$ denotes that player i's hop has already been consumed and is unavailable for the remainder of the game.

Both players initialize with $h_i = 1$. The hop action, denoted $a^{\mathrm{hop}}(c')$, enables a pawn to bypass an opponent's wall to reach a destination cell $c' \in C$. Its execution is governed by a domain-specific hop legality predicate:

$$\mathrm{LegalHop}(s_t, i, c') \in \{0, 1\}, \tag{14}$$

which evaluates to 1 if and only if:

1. player i's hop resource is unused ($h_i = 1$),

2. the hop bypasses an opponent's wall,
3. the destination cell c' is a valid board cell ($c' \in C$),
4. the destination cell is not temporarily blocked ($\tau_t(c') = 0$), and
5. the move satisfies the game's defined geometric and legal hop conditions.

The state transition for the hop resource follows an irreversible step function:

$$h_i' = \begin{cases} 0 & \text{if player } i \text{ executes } a^{\text{hop}}(c'), \\ h_i & \text{otherwise.} \end{cases} \tag{15}$$

Once $h_i = 0$, player i cannot execute hop actions for the remainder of the game.

3.6 Temporary Blocked Cells

The third modification permits players to temporarily block grid cells, rendering them impassable for pawn traversal over a fixed temporal horizon. Unlike walls, which reside on cell boundaries, temporary blocks apply directly to the vertices (cells) of the grid graph.

The configuration of temporary blocks is formalized by a duration mapping:

$$\tau_t : C \to \{0, 1, \ldots, K\}, \tag{16}$$

where C is the set of all board cells, and $K \in \mathbb{N}_{\geq 1}$ is a predefined fixed duration parameter representing the lifespan of a temporary block. For each cell $c \in C$:

- $\tau_t(c) = 0$ signifies that cell c is currently unblocked and passable.
- $\tau_t(c) = k > 0$ signifies that cell c is temporarily blocked and remains impassable for k remaining state transitions.

The duration unit is defined strictly in terms of discrete state transitions (actions). If a cell c^* is blocked by an action at time step t, it remains unavailable for movement during the next K subsequent state transitions ($t+1, \ldots, t+K$), and automatically reverts to available status ($\tau_{t+K+1}(c^*) = 0$) following the K-th subsequent transition. Since exactly one player acts at each time step, each state transition corresponds to one player action. When a cell-blocking action is executed targeting cell $c^* \in C$, its duration is initialized to $\tau_{t+1}(c^*) = K$, while all other cell durations decay by 1 at each transition:

$$\tau_{t+1}(c) = \begin{cases} K & \text{if } c = c^* \text{ (newly blocked by action } a_t), \\ \max\big(0,\, \tau_t(c) - 1\big) & \text{for all } c \in C \setminus \{c^*\}. \end{cases} \tag{17}$$

For any action that does not initiate a cell block, the duration of every cell $c \in C$ decrements monotonically: $\tau_{t+1}(c) = \max\big(0, \tau_t(c) - 1\big)$.

As an explicit rule of the environment, temporary cell blocking must preserve at least one valid path from each player's current position to that player's goal:

$$\text{PathExists}(s_{t+1}, j) = 1, \quad \forall j \in \{1, 2\}. \tag{18}$$

This requirement makes temporary blocking consistent with the path-preservation principle established for permanent walls and prevents the environment from generating unreachable goal states. Furthermore, temporary blocking actions are legally valid only on cells that are currently unoccupied ($c^* \notin \{p_1, p_2\}$) and currently unblocked ($\tau_t(c^*) = 0$).

3.7 Complete State Representation

Synthesizing the individual components, the complete environment state at time step t is fully characterized by the tuple:

$$s_t = \big(p_1, p_2, W_{1,t}, W_{2,t}, m_1, m_2, r_1, r_2, h_1, h_2, \tau_t, z_t\big) \in \mathcal{S}. \tag{19}$$

The constituents of s_t are defined as follows:

- $p_1, p_2 \in C$: current pawn positions of Player 1 and Player 2.
- $W_{1,t}, W_{2,t} \subset \mathcal{W}$: active wall sets placed by Player 1 and Player 2, defining the active wall configuration $W_t = W_{1,t} \cup W_{2,t}$.
- $m_1, m_2 \in \{0, 1, \ldots, M\}$: remaining wall inventory counts for each player, where $M = 10$.
- $r_1 \in W_{1,t} \cup \{\varnothing\}$, $r_2 \in W_{2,t} \cup \{\varnothing\}$: identity of the single replaceable wall for each player.
- $h_1, h_2 \in \{0, 1\}$: binary hop availability flags.
- $\tau_t \in \{0, 1, \ldots, K\}^{|C|}$: vector of remaining blocking durations across all cells.
- $z_t \in \mathcal{N}$: active player index controlling the transition at step t.

For any fixed board dimension N and duration parameter K, the state space $\mathcal{S}$ is strictly finite. The state s_t encodes the complete history of all strategic resources, temporal decay counters, and physical configurations necessary to determine future valid moves. Hence, the environment satisfies the Markov property:

$$\mathbb{P}(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = \mathbb{P}(s_{t+1} \mid s_t, a_t). \tag{20}$$

Because the environment is fully deterministic, the transition distribution collapses to a deterministic Dirac delta centered on the transition mapping $s_{t+1} = T(s_t, a_t)$.

3.8 Action Space

At step t, the active agent $i = z_t$ chooses an action from a state-dependent legal action set $\mathcal{A}_i(s_t) \subseteq \mathcal{A}_i$. The total action space $\mathcal{A}_i$ comprises five distinct action categories:

$$\mathcal{A}_i = \mathcal{A}^{\text{move}} \cup \mathcal{A}^{\text{place}} \cup \mathcal{A}^{\text{replace}} \cup \mathcal{A}^{\text{hop}} \cup \mathcal{A}^{\text{block}}. \tag{21}$$

The legal choices within each category in state s_t are constrained as follows:

1. **Pawn Movement ($a^{\mathbf{move}}(c') \in \mathcal{A}^{\mathbf{move}}$):** The pawn moves from p_i to an orthogonally adjacent cell $c' \in C$. The move is legal if no wall in W_t intersects the traversal edge, the target cell is not temporarily blocked ($\tau_t(c') = 0$), and c' is not occupied by the opposing player's pawn ($c' \neq p_{-i}$).
2. **Wall Placement ($a^{\mathbf{place}}(w) \in \mathcal{A}^{\mathbf{place}}$):** The player selects a wall $w \in \mathcal{W} \setminus W_t$. This is legal if and only if $m_i > 0$, w does not intersect any existing wall in W_t, and path preservation holds: $\text{PathExists}(s_t \oplus w, j) = 1$ for all $j \in \mathcal{N}$, where $s_t \oplus w$ denotes the tentative state with w added.
3. **Wall Replacement ($a^{\mathbf{replace}}(w') \in \mathcal{A}^{\mathbf{replace}}$):** Available if and only if $r_i \neq \varnothing$. The player repositions r_i to a new location $w' \in \mathcal{W} \setminus (W_t \setminus \{r_i\})$. The action is legal if w' does not intersect any wall in $W_t \setminus \{r_i\}$ and path preservation is satisfied: $\text{PathExists}((s_t \ominus r_i) \oplus w', j) = 1$ for all $j \in \mathcal{N}$.
4. **One-Time Hop ($a^{\mathbf{hop}}(c') \in \mathcal{A}^{\mathbf{hop}}$):** Available if and only if $\text{LegalHop}(s_t, i, c') = 1$.
5. **Temporary Cell Blocking ($a^{\mathbf{block}}(c) \in \mathcal{A}^{\mathbf{block}}$):** The player blocks cell $c \in C$. This action is legal if $\tau_t(c) = 0$, $c \notin \{p_1, p_2\}$, and the temporary obstruction preserves reachable paths: $\text{PathExists}(s_t \oplus_\tau c, j) = 1$ for all $j \in \mathcal{N}$.

Action availability is dynamic: the state-dependent action set $\mathcal{A}_i(s_t)$ contracts as inventory is depleted ($m_i = 0$), the hop is consumed ($h_i = 0$), or cells and wall locations become obstructed.

3.9 State Transition Function

The deterministic transition function is defined strictly over legal state-action pairs:

$$T : \big\{(s,a) \;\big|\; s \in \mathcal{S},\ a \in \mathcal{A}_{z(s)}(s)\big\} \to \mathcal{S}, \tag{22}$$

where $z(s) \in \mathcal{N}$ identifies the active player in state s, ensuring that only actions from that player's state-dependent legal action set $\mathcal{A}_{z(s)}(s)$ can be executed. The transition mapping uniquely determines the subsequent state $s_{t+1} = T(s_t, a_t)$.

Let $i = z_t$ be the active player and $-i = 3 - i$ be the opponent. For all non-blocking actions, remaining temporary block durations decay according to $\tau_{t+1}(c) = \max\big(0, \tau_t(c) - 1\big)$, while for temporary blocking actions, durations update according to Equation (17). The active turn toggles to $z_{t+1} = -i$, unless a terminal state is reached. The component-wise updates for each action category are:

- If $a_t = a^{\text{move}}(c')$:

$$p'_i = c', \quad p'_{-i} = p_{-i}, \quad W'_j = W_j, \quad m'_j = m_j, \quad r'_j = r_j, \quad h'_j = h_j \quad (\forall j \in \mathcal{N}). \tag{23}$$

- If $a_t = a^{\text{place}}(w)$:

$$W'_i = W_i \cup \{w\}, \quad W'_{-i} = W_{-i}, \quad m'_i = m_i - 1, \quad m'_{-i} = m_{-i}, \tag{24}$$
$$r'_i = w, \quad r'_{-i} = r_{-i}, \quad p'_j = p_j, \quad h'_j = h_j \quad (\forall j \in \mathcal{N}). \tag{25}$$

- If $a_t = a^{\text{replace}}(w')$:

$$W'_i = (W_i \setminus \{r_i\}) \cup \{w'\}, \quad W'_{-i} = W_{-i}, \quad m'_j = m_j \quad (\forall j \in \mathcal{N}), \tag{26}$$
$$r'_i = w', \quad r'_{-i} = r_{-i}, \quad p'_j = p_j, \quad h'_j = h_j \quad (\forall j \in \mathcal{N}). \tag{27}$$

- If $a_t = a^{\text{hop}}(c')$:

$$p'_i = c', \quad p'_{-i} = p_{-i}, \quad h'_i = 0, \quad h'_{-i} = h_{-i}, \quad W'_j = W_j, \quad m'_j = m_j, \quad r'_j = r_j \quad (\forall j \in \mathcal{N}). \tag{28}$$

- If $a_t = a^{\text{block}}(c^*)$:

$$\tau_{t+1}(c^*) = K, \quad \tau_{t+1}(c) = \max\big(0,\ \tau_t(c) - 1\big)\ (\forall c \neq c^*), \tag{29}$$

  with all pawn positions, wall sets, inventories, replaceable wall pointers, and hop resources remaining invariant.

3.10 Terminal States and Winning Condition

The game terminates immediately when either player successfully advances their pawn into their designated goal region. The set of terminal states is defined as:

$$\mathcal{S}_{\text{terminal}} = \big\{s \in \mathcal{S} \;\big|\; p_1 \in G_1 \ \vee\ p_2 \in G_2\big\}. \tag{30}$$

Because turn alternation is strictly sequential and players begin in non-terminal states, exactly one player achieves the winning condition on their active turn. The winner of the game is determined unambiguously by:

$$\text{Winner}(s) = \begin{cases} 1 & \text{if } p_1 \in G_1, \\ 2 & \text{if } p_2 \in G_2. \end{cases} \tag{31}$$

The action validation constraints and deterministic transition function T guarantee that illegal configurations—such as overlapping pawns, intersecting walls, or invalid topologies violating PathExists—are unreachable from any legal initial configuration.

Episode truncation, repetition handling, and maximum training horizon, if required for reinforcement-learning experiments, will be specified separately in the experimental setup.

3.11 Reward Function

Modified Wallz is a strictly competitive zero-sum game:

$$R_1(s, a, s') + R_2(s, a, s') = 0, \quad \forall (s, a, s') \in \mathcal{S} \times \mathcal{A} \times \mathcal{S}. \tag{32}$$

We formulate the base game objective using canonical sparse terminal rewards:

$$R_i(s, a, s') = \begin{cases} +1 & \text{if } s' \in \mathcal{S}_{\text{terminal}} \text{ and } \text{Winner}(s') = i, \\ -1 & \text{if } s' \in \mathcal{S}_{\text{terminal}} \text{ and } \text{Winner}(s') = -i, \\ 0 & \text{if } s' \notin \mathcal{S}_{\text{terminal}}. \end{cases} \tag{33}$$

This formulation directly reflects the underlying win/loss objective of the zero-sum game. Potential intermediate reward shaping (such as potential-based distance-to-goal heuristics) may be applied during reinforcement learning experimentation to mitigate credit assignment challenges, but remains strictly separated from the formal mathematical specification of the underlying game.

3.12 Observation Model

The game exhibits perfect information and complete observability. Each player observes the identical, complete environment state:

$$o_i(s_t) = s_t, \quad \forall i \in \mathcal{N}. \tag{34}$$

At every decision step, both agents have instantaneous, unoccluded access to:

- the exact coordinate locations of both pawns (p_1, p_2),
- the complete active wall configuration $(W_{1,t}, W_{2,t})$,
- the remaining wall inventories (m_1, m_2),
- the replaceable wall identities (r_1, r_2),
- the binary hop resource status (h_1, h_2),
- the spatial duration profile of all temporary blocked cells (τ_t), and
- the current turn indicator (z_t).

The absence of hidden variables or partial observability confirms that Modified Wallz is a fully observable Markov game. The modifications systematically enrich the state space beyond classical Wallz by incorporating temporal decay (τ_t), discrete resource counters (m_i, h_i), and history-dependent wall replacement pointers (r_i). These elements induce deep temporal and spatial coupling, wherein present actions dynamically alter the availability of future tactical actions for both agents.

4 Proposed Solutions

To establish rigorous benchmarks for evaluating multi-agent reinforcement learning (MARL) policies in subsequent stages of this work, we propose a hierarchy of non-learning reference methods. Because Modified Wallz is a deterministic, sequential, zero-sum game of perfect information, it lends itself to classical heuristic evaluation, depth-bounded adversarial search, and exact game-tree analysis. These baselines are designed to provide interpretable, structured reference points across varying degrees of computational complexity and lookahead depth. In this section, we formulate the proposed non-learning baselines and outline the planned protocol for comparative evaluation.

4.1 Heuristic Baseline

We propose a deterministic, greedy heuristic agent whose policy is guided by topological distance metrics over the traversable grid graph. Let $d_i(s)$ denote the length of the shortest unblocked path from player i's current pawn position p_i to its target goal set G_i in state $s \in \mathcal{S}$, computed over the cell adjacency structure induced by the active walls and currently unexpired blocked cells ($\tau_t(c) > 0$). When no valid path exists or if the player is trapped, $d_i(s) = \infty$; however, the path-preservation constraint $\text{PathExists}(s, i) = 1$ ensures that $d_i(s) < \infty$ for all legal states.

We define the relative spatial progress heuristic for player i as:

$$H_i(s) = d_{-i}(s) - d_i(s), \tag{35}$$

where $-i = 3 - i$ denotes the opposing player. The value $H_i(s)$ captures the net path advantage of player i: positive values indicate that player i is closer to victory than its opponent, whereas negative values reflect an opponent lead.

For any decision step with active player $i = z(s)$, the proposed heuristic agent will evaluate all candidate legal actions $a \in \mathcal{A}_i(s)$ by inspecting the immediate successor state $s' = T(s, a)$. The planned action-selection policy is formalized as:

$$a^* = \arg\max_{a \in \mathcal{A}_i(s)} H_i\big(T(s, a)\big), \tag{36}$$

with ties broken deterministically according to a predefined priority ordering over action categories.

Crucially, this evaluation will encompass all five legal action categories defined in Section 3.8:

- **Pawn movement (a^{move}):** Directly decreases $d_i(s')$ by advancing the pawn along an orthogonal trajectory toward G_i.
- **Wall placement (a^{place}):** Introduces structural barriers that can lengthen the opponent's shortest path $d_{-i}(s')$, while strictly maintaining $d_i(s') < \infty$.
- **Wall replacement (a^{replace}):** Repositions the active wall r_i to reconfigure spatial obstructions, permitting dynamic adjustments to $d_{-i}(s')$ without depleting inventory m_i.
- **One-time hop (a^{hop}):** Bypasses an intervening opponent wall to achieve an immediate reduction in $d_i(s')$, permanently setting $h_i' = 0$.
- **Temporary cell blocking (a^{block}):** Imposes a transient obstruction on cell c^*, temporarily forcing the opponent onto a longer detour and increasing $d_{-i}(s')$.

This heuristic agent is intended to serve as a fast, computationally inexpensive, and intuitive first-order baseline against which more sophisticated decision-making architectures can be compared.

4.2 Search-Based Baseline

While the greedy heuristic baseline considers only immediate successor states, competitive decision-making often demands multi-step adversarial anticipation. Because Modified Wallz is deterministic and strictly zero-sum, minimax search over the game tree provides a principled framework for evaluating future counteractions.

We propose a depth-limited minimax agent with heuristic leaf evaluation. Let $d \in \mathbb{N}_0$ denote the remaining search depth, and let $D \in \mathbb{N}_{\geq 1}$ be the maximum lookahead depth parameter. The minimax value function $V_i(s, d)$ for player i in state s is defined recursively as:

$$V_i(s, d) = \begin{cases} U_i(s), & \text{if } s \in \mathcal{S}_{\text{terminal}} \text{ or } d = 0, \\ \max\limits_{a \in \mathcal{A}_i(s)} V_i\big(T(s, a),\, d - 1\big), & \text{if } z(s) = i, \\ \min\limits_{a \in \mathcal{A}_{-i}(s)} V_i\big(T(s, a),\, d - 1\big), & \text{if } z(s) = -i, \end{cases} \tag{37}$$

where $U_i(s)$ is the static evaluation function at terminal states or search horizons:

$$U_i(s) = \begin{cases} +\infty, & \text{if } s \in \mathcal{S}_{\text{terminal}} \text{ and } \text{Winner}(s) = i, \\ -\infty, & \text{if } s \in \mathcal{S}_{\text{terminal}} \text{ and } \text{Winner}(s) = -i, \\ H_i(s), & \text{if } s \notin \mathcal{S}_{\text{terminal}} \text{ and } d = 0. \end{cases} \tag{38}$$

At any decision state s with active player $i = z(s)$, the proposed agent selects the action maximizing the backed-up minimax value over a lookahead horizon of D:

$$a^* = \arg\max_{a \in \mathcal{A}_i(s)} V_i\big(T(s,a),\, D-1\big). \tag{39}$$

During search, the agent will explore branching paths across all five legal action categories. To mitigate the combinatorial growth of the game tree, we plan to implement **Alpha-Beta pruning**. Alpha-Beta pruning maintains running bounds $[\alpha, \beta]$ representing the minimum score the maximizing player is assured of and the maximum score the minimizing player can limit the opponent to, respectively. By pruning branches that cannot influence the final root decision, Alpha-Beta search achieves substantial reductions in explored nodes while guaranteeing mathematical equivalence to the full minimax value. The lookahead depth D will be treated as an experimental parameter to be selected during subsequent benchmarking based on computational budget constraints.

4.3 Exact Game Analysis for Small Configurations

For sufficiently small board dimensions and restricted resource budgets, it becomes computationally feasible to construct the complete reachable game tree. In such configurations, exact backward induction can determine the true game-theoretic value of every reachable state under optimal play.

Because Modified Wallz possesses a finite state space $\mathcal{S}$, deterministic transitions T, sequential turn structure, and perfect information, the optimal value $V_i^*(s)$ satisfies the unbounded minimax fixed-point formulation:

$$V_i^*(s) = \begin{cases} R_i(s), & \text{if } s \in \mathcal{S}_{\text{terminal}}, \\ \max\limits_{a \in \mathcal{A}_i(s)} V_i^*\big(T(s,a)\big), & \text{if } z(s) = i, \\ \min\limits_{a \in \mathcal{A}_{-i}(s)} V_i^*\big(T(s,a)\big), & \text{if } z(s) = -i. \end{cases} \tag{40}$$

Here, $R_i(s) \in \{+1, -1\}$ denotes the terminal win/loss outcome.

We propose an exact game-tree solver using retrograde analysis and exhaustive minimax traversal to serve as an absolute **ground-truth reference**. This solver will enable direct, rigorous verification of:

1. the degree of suboptimality in heuristic-based decisions,
2. the convergence properties and approximation accuracy of depth-limited search as depth D increases, and
3. the policy fidelity of learned reinforcement learning agents against true game-theoretic equilibria.

However, the state space of Modified Wallz expands rapidly due to the combined combinatorial configurations of:

- pawn coordinate pairs $(p_1, p_2) \in C^2$,
- active wall combinations $W_t \subset \mathcal{W}$,
- remaining inventory counters $(m_1, m_2) \in \{0, \dots, M\}^2$,
- replaceable wall identifiers (r_1, r_2),
- binary hop availability flags $(h_1, h_2) \in \{0, 1\}^2$,
- discrete countdown vectors for temporary blocked cells $\tau_t \in \{0, \dots, K\}^{|C|}$, and

- turn alternating variables $z_t \in \{1, 2\}$.

Consequently, exact game-tree computation will be restricted to miniature configurations (such as reduced grid sizes N and bounded wall inventories) that are verified to be computationally tractable during the implementation phase.

4.4 Planned Baseline Evaluation

The proposed reference methods establish an explicit evaluation ladder with three distinct tiers of reasoning capability:

1. **Heuristic Agent (Tier 1):** Immediate, myopic spatial reasoning based solely on shortest-path differential $H_i(s')$.
2. **Search-Based Agent (Tier 2):** Multi-step adversarial reasoning via depth-limited minimax with Alpha-Beta pruning, projecting game dynamics up to horizon D.
3. **Exact Solver (Tier 3):** Provably optimal game-theoretic reference via complete backward induction on computationally tractable sub-configurations.

These baseline agents will subsequently be evaluated against one another and against trained MARL policies under identical environment dynamics, legal-action constraints, and path-preservation rules. The planned evaluation protocol will measure:

- **Competitive Outcome:** Win/loss outcomes across alternating starting positions to account for first-player advantage.
- **Trajectory Efficiency:** Average game length (measured in elapsed state transitions until goal attainment).
- **Computational Burden:** Wall-clock time, node evaluations, and memory footprint per decision step.
- **Policy Fidelity:** Frequency of agreement between agent action selections and optimal minimax actions in configurations where exact ground-truth solutions are available.

Furthermore, the evaluation framework is planned to systematically vary experimental parameters, including grid dimension N, temporary blocking duration K, and the selective enablement of individual modifications (reusable walls, one-time hops, and temporary blocks). This parameter ablation will isolate how each distinct mechanism affects computational complexity, search depth sensitivity, and tactical diversity.

This multi-tiered evaluation foundation directly prepares the ground for formulating and benchmarking the multi-agent reinforcement learning methodology to be developed in subsequent stages of this project.

4.5 MARL Methodology

This subsection formulates the proposed learning approach for Modified Wallz as a two-agent, turn-based, zero-sum Markov game, building directly upon the formal game specification established in Section 3.

4.5.1 MARL Formulation

The learning environment is formulated as an interaction between two competing agents with strictly opposing objectives. Rather than reformulating the underlying game dynamics, the MARL setup directly adopts the formal specification from Section 3. In particular:

- Both agents have complete access to the current environment state.
- Only the currently active player $z_t \in \{1, 2\}$ selects and executes an action at each discrete timestep t.
- The environment deterministically transitions to the successor state $s_{t+1} = T(s_t, a_t)$.
- Players strictly alternate decision-making turns.

- The learning objective for each agent is to maximize its expected game outcome against an adaptive opponent.

In accordance with the underlying environment model, the learning framework maintains perfect information, deterministic transitions, alternating turns, and a strictly zero-sum terminal objective, without introducing simultaneous moves, partial observability, environmental stochasticity, or draw outcomes.

4.5.2 Observation Representation

Consistent with the perfect-information nature of Modified Wallz and the observation model defined in Section 3.12, each agent receives the complete environment state as its observation:

$$o_i(s_t) = s_t. \tag{41}$$

As formally defined in Section 3.7, the complete state vector $s_t = (p_1, p_2, W_{1,t}, W_{2,t}, m_1, m_2, r_1, r_2, h_1, h_2, \tau_t, z_t)$ encapsulates both the physical board configuration and the strategic resources available to the two players. Here, $W_{1,t}$ and $W_{2,t}$ denote the active walls placed by Players 1 and 2, respectively, with $W_t = W_{1,t} \cup W_{2,t}$ representing the complete active wall configuration. The observation vector thus incorporates:

- both pawn positions $p_1, p_2 \in C$;
- the active wall sets $W_{1,t}$ and $W_{2,t}$, including their respective ownership and orientations;
- the remaining unused wall inventories (m_1, m_2);
- the identifiers of each player's most recently placed (repositionable) wall (r_1, r_2);
- the binary hop availability flags (h_1, h_2);
- the temporary blocked-cell countdown mapping τ_t; and
- the active turn indicator $z_t \in \{1, 2\}$.

Providing the full state representation ensures that the learned policy operates with complete transparency over all physical, temporal, and inventory constraints governing the game.

4.5.3 Action Representation and Legal-Action Masking

Modified Wallz features a state-dependent discrete action space, as the set of legally permissible actions $\mathcal{A}_i(s_t)$ varies substantially across different board configurations. At any decision state, the five action categories defined in Section 3.8 are:

1. standard pawn movement: $\mathcal{A}_{\text{move}}(s_t, i)$;
2. wall placement: $\mathcal{A}_{\text{wall}}(s_t, i)$;
3. replacement of the player's most recently placed wall: $\mathcal{A}_{\text{replace}}(s_t, i)$;
4. execution of the one-time hop: $\mathcal{A}_{\text{hop}}(s_t, i)$; and
5. temporary cell blocking: $\mathcal{A}_{\text{block}}(s_t, i)$.

For the MARL implementation, we propose a fixed discrete action encoding that spans all representable action instances in the game universe, paired with an exact legal-action mask. Formally, for any candidate action a in the discrete action universe, the mask $M(s_t, a)$ is defined as:

$$M(s_t, a) = \begin{cases} 1, & \text{if } a \in \mathcal{A}_i(s_t), \\ 0, & \text{if } a \notin \mathcal{A}_i(s_t). \end{cases} \tag{42}$$

Under this masking scheme, only actions satisfying $M(s_t, a) = 1$ may be selected by the agent. By explicitly constraining the policy or value-selection operator to the masked subset of valid actions, the learning algorithm

ignores invalid actions entirely during decision-making, rather than requiring the agent to learn via negative penalties that illegal moves are impermissible. This formulation is particularly advantageous for Modified Wallz, where legality constraints—including pawn adjacency, wall overlaps, path-preservation requirements, recency-based wall replacement, hop conditions, and temporary block validity—depend intimately on the current board state.

4.5.4 Reward Design

The reward function directly follows the formal sparse zero-sum specification defined in Section 3.11:

$$R_i(s_t, a_t, s_{t+1}) = \begin{cases} +1, & \text{if player } i \text{ wins,} \\ -1, & \text{if player } i \text{ loses,} \\ 0, & \text{otherwise.} \end{cases} \tag{43}$$

The reward signal is intentionally kept sparse and strictly aligned with the true game objective of reaching the goal region. At this stage, no reward shaping is introduced; specifically, intermediate distance-based rewards, penalties for wall expenditure, or handcrafted heuristic incentives are omitted. This sparse reward formulation ensures that the learning agent is encouraged to discover emergent strategic behaviors and tactical trade-offs directly from the fundamental game objective, avoiding biases toward manually engineered preferences.

4.5.5 Self-Play Training

We propose self-play as the primary training paradigm for the learning agents. Rather than training exclusively against fixed heuristic policies or static opponents, the agents will repeatedly play Modified Wallz against policies generated from the same learning process. An interaction trajectory is conceptually represented as a sequence of alternating decision epochs:

$$s_0 \xrightarrow{a_0} s_1 \xrightarrow{a_1} s_2 \xrightarrow{a_2} \cdots \xrightarrow{a_{T-1}} s_T, \tag{44}$$

where each transition advances the board state deterministically and alternates the active player index z_t. An episode terminates at step T when a pawn reaches its designated goal region, emitting the terminal sparse rewards.

The purpose of self-play is to provide an evolving curriculum wherein each agent is exposed to progressively stronger, co-adapting counter-strategies. As training progresses, the opposing policy provides an adaptive challenge, driving the discovery of sophisticated offensive paths, timely wall placements, counter-blocking maneuvers, and tactical hop usage.

4.5.6 Value-Based Learning Approach

For the initial MARL implementation, we propose a DQN-style value-based learning approach augmented with legal-action masking. In this framework, a parameterized state-action value function is learned:

$$Q_\theta(s, a), \tag{45}$$

where θ denotes the learnable parameters of the function approximator. At each decision state s_t, the active agent selects the legal action that maximizes the estimated value:

$$a_t^* = \arg \max_{a \in \mathcal{A}_i(s_t)} Q_\theta(s_t, a). \tag{46}$$

The legal-action mask ensures that this maximization is evaluated strictly over the set of legally valid actions $\mathcal{A}_i(s_t)$, setting the effective value of illegal actions to $-\infty$ during selection.

This value-based formulation is chosen for several reasons:

- the environment operates with discrete, alternating turns;
- the action space, while state-dependent, is discrete;

- state transitions are fully deterministic;
- the environment exhibits full observability of the complete game state;
- the game has a well-defined zero-sum terminal objective;
- value-based methods are comparatively straightforward to implement, stabilize, and interpret; and
- learned state-action values $Q_\theta(s, a)$ provide a transparent basis for comparison against the value estimates and action choices of the heuristic and minimax baselines.

We do not assert that this value-based formulation is theoretically optimal for the game. Consistent with the current planning and ideation phase, specific architectural dimensions, hyperparameter configurations, replay-buffer mechanisms, target-network update schedules, exploration strategies, and training durations are not predefined; these operational choices will be finalized during the implementation phase based on the concrete action-space representation and available computational resources.

4.6 Planned MARL Evaluation

Following implementation, the trained MARL policies will be evaluated through a structured experimental framework designed to benchmark competitive capability, strategic quality, and sensitivity to environmental parameters.

4.6.1 Comparison Against Baselines

The performance of the learned MARL agents will be directly evaluated against the baseline agents established in Sections 4.1–4.3:

$$\begin{aligned} &\text{MARL} \quad \text{vs} \quad \text{Heuristic Baseline}, \\ &\text{MARL} \quad \text{vs} \quad \text{Minimax Baseline (varying depth } D), \end{aligned}$$

and, on computationally tractable miniature configurations:

$$\text{MARL} \quad \text{vs} \quad \text{Exact Game-Tree Solver}.$$

These comparative match-ups will determine whether the learned policy discovers effective competitive strategies, and how its tactical decisions compare with handcrafted spatial heuristics, multi-step tree search, and provably optimal game-theoretic play. In accordance with the prospective nature of this report, no specific outcome or performance superiority is predicted prior to empirical execution.

4.6.2 Exact-Game Policy Comparison

For miniature board configurations where backward induction and exhaustive game-tree analysis remain computationally tractable, the decision policy learned by the MARL agent will be compared directly against the provably optimal actions derived from the exact solver. We define the policy-agreement metric conceptually as:

$$\text{Agreement} = \frac{\text{number of states where MARL selects an optimal action}}{\text{number of evaluated states}}. \tag{47}$$

In game states where multiple actions yield identical game-theoretic values under the exact minimax solution, an action chosen by the MARL policy will be considered in agreement whenever it belongs to the set of optimal actions. This metric will quantify the degree of equilibrium fidelity achieved by the learned policy on small problem instances.

4.6.3 Ablation Study

To isolate the individual and combined strategic effects of the three novel mechanics introduced in Modified Wallz, we plan an ablation study across five distinct environment configurations:

1. **Original Wallz:** standard pawn movements and wall placements with non-reusable walls;
2. **Wallz with Reusable Walls:** incorporating the repositioning mechanism for the most recently placed wall;
3. **Wallz with One-Time Hop:** enabling a single wall-hop maneuver per player under legal conditions;
4. **Wallz with Temporary Blocked Cells:** introducing the K-step cell-blocking mechanism; and
5. **Full Modified Wallz:** the complete environment incorporating all three mechanics concurrently.

This ablation protocol will systematically examine how each individual mechanism alters policy behavior, tactical complexity, and learned defensive versus offensive trade-offs.

4.6.4 Parameter Variation

The planned experimental campaign will systematically vary the primary environmental parameters:

$$N = \text{board dimension } (N \times N \text{ discrete grid}),$$
$$K = \text{temporary block duration (number of elapsed state transitions)}.$$

Varying these parameters will enable investigation into how increasing spatial complexity and temporal constraint horizons affect:

- sample efficiency and learning stability during training;
- competitive game outcomes against baselines;
- average trajectory length and game duration;
- computational and memory requirements per training epoch; and
- qualitative policy adaptations (e.g., preference for spatial trapping versus rapid advancement).

Concrete numerical choices for N, K, and the total training budget will be selected during implementation based on empirical computational feasibility.

4.6.5 Evaluation Metrics

To maintain consistency with the baseline evaluation protocol in Section 4.4, the MARL evaluation will employ the following core metrics:

- **Competitive Performance:** Win/loss rate across evaluation matches, evaluated symmetrically with alternating starting players to eliminate first-mover bias.
- **Trajectory Efficiency:** Average game length, measured by the total number of elapsed state transitions until goal attainment.
- **Computational and Training Cost:** Wall-clock training duration, inference and decision-making time per turn, and peak memory utilization.
- **Policy Quality:** Frequency of agreement between MARL action selections and exact optimal actions on configurations where ground-truth solutions are available.

These standardized metrics will provide a comprehensive, multi-dimensional assessment of learned agent capability without introducing undefined or unmodeled outcomes.

5 Contribution

The individual contributions of the team members to this project report are summarized in Table 2.

Table 2: Team member contribution allocation.

Member	**Contribution**
Vedant Bondekar	Game design and modification; report writing.
Dhruv Meshram	Report writing; ideation and brainstorming.
Abhishek Tagalpallewar	Game design and modification; custom game making.
Chaitanya Chaudhary	Game design and modification; literature review.

References

[1] Michael L. Littman. Markov games as a framework for multi-agent reinforcement learning. In *Proceedings of the Eleventh International Conference on Machine Learning*, pages 157–163. Morgan Kaufmann, 1994.

[2] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. A comprehensive survey of multiagent reinforcement learning. *IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews*, 38(2):156–172, 2008.

[3] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *Studies in Systems, Decision and Control*, 325:321–384, 2021.

[4] Sven Gronauer and Klaus Diepold. Multi-agent deep reinforcement learning: A survey. *Artificial Intelligence Review*, 55:895–943, 2022.

[5] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, 2 edition, 2018.